

forge-harness: Engineering Methods for Robust AI Collaboration Harnesses

[Author]

Independent Researcher · [email placeholder]

Draft v0.5 · 2026-05-26 · arXiv cs.SE / cs.AI

Abstract

AI collaboration harnesses — structured environments of prompts, skills, agents, and memory files that orchestrate AI-assisted work — have become primary determinants of AI output quality. Empirical analysis finds that 98.4% of Claude Code session content is harness infrastructure rather than direct task instructions [VILA-Lab, 2026]. Despite this, harness engineering lacks principled validation and evolution methods: harnesses are typically built and evaluated by the same practitioner, documentation drifts from implementation, phantom claims accumulate in AI-generated content, and session learnings are lost between working sessions.

We present **forge-harness**, an open harness engineering toolkit that addresses these gaps through four complementary methods: (1) **steel-quench**, a multi-wave adversarial validation framework that separates attack from defense to systematically surface structural defects; (2) **source-grounding-audit**, a phantom claim detection method that demands file-path, commit-hash, or measured-value evidence for every factual claim in harness content; (3) **harvest-loop**, a session-to-harness self-evolution pipeline that transforms session learnings into validated skill upgrades without accumulating technical debt; and (4) **sim-conductor**, a pre-deployment simulation framework that validates harness transferability before non-owner adoption.

Applied to forge-harness itself — a self-verification scenario — the four methods resolved 10 S/A-grade structural defects, detected 3 phantom capability claims, absorbed 5 external session contributions as validated skill upgrades, and confirmed multi-user autonomous operation across three independent deployments. The methodology independently converged on the same outer-loop architecture as harness-evolver [Christi, 2026] and the Stanford Meta-Harness [arXiv:2603.28052], validating the structural approach across independent implementations. forge-harness is available at <https://github.com/chrono-code/forge-harness>.

1. Introduction

AI collaboration harnesses have evolved from simple prompt templates into layered systems comprising memory files, skill libraries, agent dispatch protocols, domain knowledge bases, and behavioral rules. As these harnesses grow in complexity, four specific failure modes recur across practitioner deployments:

F1 — Self-referential validation: The practitioner who builds the harness also evaluates it. A harness that passes its own internal checks may harbor defects visible only to an adversarial evaluator. This is structurally analogous to the penetration-testing problem in software security.

F2 — Phantom claim accumulation: AI-generated harness content (skill documentation, capability descriptions, validation evidence) frequently references files, measurements, or capabilities that do not exist. These "phantom claims" pass informal review and accumulate silently, eroding the reliability of harness content over time.

F3 — Session learning loss: Valuable patterns discovered during working sessions — failure modes caught, successful approaches confirmed, defects identified — are rarely absorbed into the harness systematically. They exist in session transcripts and are lost when context resets.

F4 — Unvalidated transfer: A harness developed by one practitioner may exhibit silent failures when adopted by others — different vocabulary triggering skills, different mental models of skill boundaries, different baseline contexts. Pre-deployment validation of transferability is typically absent.

We introduce **forge-harness**, a toolkit addressing each failure mode through a dedicated method. Our contributions are:

- Four complementary harness engineering methods addressing F1–F4 (Section 3)
- The **Sandboxed Adversary principle**, explaining structural convergence of adversarial validation (Section 3.1)
- The **phantom claim taxonomy**, classifying three categories of AI-generated false claims in harness content (Section 3.2)
- The **synthesizer gate**, a cross-validation step preventing both over-addition and under-addition in harness evolution (Section 3.3)
- Empirical application to forge-harness itself, with independent architectural convergence evidence (Section 4)

2. Background and Related Work

2.1 The Harness Infrastructure Problem

Chen et al. [2026] analyzed 10,000 Claude Code sessions and found that 98.4% of token content constitutes harness infrastructure — CLAUDE.md rules, memory files, skill invocations, context bridges — rather than direct task instructions [arXiv:2604.14228]. Several concurrent works confirm this framing. "Code as Agent Harness" [arXiv:2605.18747] presents a 43-author study positioning harness code as the primary determinant of agent performance. Sylph.AI's "The Last Harness You'll Ever Build" [arXiv:2604.21003] introduces a self-evolving harness architecture. The Stanford IRIS Lab Meta-Harness [arXiv:2603.28052] demonstrates an outer-loop system achieving +7.7 benchmark points at 4× fewer tokens through automated harness code optimization.

2.2 Adversarial Testing and Hallucination Detection

Red-teaming [Perez et al., 2022] and constitutional AI [Bai et al., 2022] apply adversarial pressure to AI model outputs. Our work applies analogous pressure one layer up: to the harness infrastructure itself, not to model outputs. Hallucination detection in AI outputs [Maynez et al., 2020; Ji et al., 2023] focuses on factual grounding in generated text. source-grounding-audit extends this to harness content specifically, where phantom claims are particularly consequential because harness documentation is treated as ground truth by the orchestrating AI.

2.3 Self-Evolving Systems

Reflexion [Shinn et al., 2023] and Self-Refine [Madaan et al., 2023] enable agents to revise their own outputs through verbal feedback. harvest-loop applies analogous self-refinement at the harness level — not to individual agent outputs but to the persistent skill infrastructure across sessions. The key distinction is that harvest-loop targets cross-session knowledge accumulation rather than within-session output revision.

2.4 Positioning vs. Automated Harness Optimization

harness-evolver [Christi, 2026] implements the Meta-Harness pattern in a 7-stage automated loop requiring LangSmith instrumentation and Python infrastructure, targeting automated optimization of harness code at benchmark scale. forge-

harness targets a complementary layer: harness knowledge validation and evolution for practitioners who prioritize human-approved architectural gates and zero additional infrastructure. These are not competing approaches — automated code optimization (harness-evolver) and principled knowledge engineering (forge-harness) address different layers of the same problem.

3. Methods

3.1 steel-quench — Adversarial Validation (F1)

steel-quench addresses F1 through strict structural separation of attack and defense across convergence rounds.

Wave 1 — Devil Attack: An adversarial agent (devil) operates in isolation with access only to static harness files. It applies five mandatory attack angles:

#	Angle	Core Question
1	Existence justification	"Why this structure? Is a simpler alternative available?"
2	Documentation-code alignment	"Do documented and actual behaviors match?"
3	Bus factor	"Can operations continue without the primary maintainer?"
4	Platform obsolescence	"Does the architecture survive ecosystem changes?"
5	Self-referential structure	"Is there a closed circuit where the system evaluates itself?"

Each finding is classified **S** (immediate blocker), **A** (required before deployment), or **B** (improvement recommended). Abstract critique without file-level citation is rejected.

Wave 2 — Defense: Defense draws on three principles: (1) *external evidence precedence* — living system history that the isolated devil cannot observe; (2) *implementation priority* — a concrete fix is stronger than a defensive argument; (3) *explicit residual risk* — unresolved defects are named and carried forward, not silently dropped.

The Sandboxed Adversary Principle: The devil operates in an information sandbox — it sees static artifacts while the defense can cite dynamic evidence (adoption rates, external PRs, multi-project history). As Waves deepen, the devil exhausts its structurally accessible attack surface. This makes the convergence criterion (zero new S-grade) theoretically grounded: it marks the point at which the adversary's accessible attack space is exhausted.

Wave 3+ — Convergence: Convergence is declared when no new S-grade blockers appear. A/B-grade residual items are captured as named technical debt.

Wave 4 — Meta-Aware Adversary: An optional depth extension in which the devil is told it is an AI with hallucination risk, context window limits, and tool dependencies — and is instructed to exploit these as attack vectors. Five AI-specific angles apply: API single point of failure, context collapse, prompt injection exposure, hallucination-contaminated defense claims, and tool dependency lock-in.

3.2 source-grounding-audit — Phantom Detection (F2)

source-grounding-audit addresses F2 through mandatory evidence citation for factual claims in harness content.

Phantom Claim Taxonomy: Three categories of phantom claims in AI-generated harness content:

- **File phantoms:** A referenced file, function, or configuration value does not exist at the cited path.

- **Capability phantoms:** A documented skill or agent behavior has never been implemented or verified in cold-start execution.
- **Measurement phantoms:** A cited metric or performance figure has no corresponding measurement artifact — the figure is LLM-reconstructed plausibility.

Detection Method: For each factual claim, a two-step check applies: (1) *existence check* — does the referenced artifact exist? (2) *origin check* — is the claim derivable from a non-LLM source (file contents, commit history, external measurement)? Claims failing either check are flagged as phantom candidates.

The Defense Evidence Standard: source-grounding-audit establishes that steel-quench Wave 2 defense claims must cite an original file path, commit hash, or measured value. "LLM-assessed" is explicitly not accepted as defense evidence, preventing hallucination contamination from propagating through Wave 2 defense arguments.

3.3 harvest-loop — Self-Evolution Pipeline (F3)

harvest-loop addresses F3 through a structured pipeline transforming session artifacts into validated harness skill upgrades:

```

Session end
  ↓
[Step 0] Regression Guard – does anything conflict with a validated skill?
  ↓
[Step 1] field-harvest – extract patterns from session diff/outputs (3+ required)
  ↓
[Step 2] contention-layer – candidate ↔ existing skills (overlap/gap/conflict)
  ↓ (parallel)
[Step 3a] devil-advocate          [Step 3b] innovator
  Attack existing skills          Propose new skills (Done-When required)
  ↓
[Step 3.5] synthesizer gate ← key innovation
  Cross-validate devil × innovator:
  · S-attack + matching proposal → HIGH
  · S-attack + no proposal       → HIGH (existing skill weakness)
  · No attack + proposal only    → MED (unvalidated; defer)
  · Attack invalidates proposal → REJECT
  ↓
[Step 4] harness-doctor – Done-When exists? overlap <70%? no self-reference?
  ↓
[Step 5] verify-bidirectional – bidirectional link consistency
  ↓
User approval gate → implement or archive

```

The Synthesizer Gate: Step 3.5 cross-validates devil-advocate attack results against innovator proposals before any skill is created or modified. Without this gate, attacks may invalidate the premises of proposed enhancements without the proposer being aware. The synthesizer gate forces explicit resolution, producing a ranked candidate list.

Contention as Signal: contention-layer treats skill conflicts as generative signals rather than problems to eliminate. When two existing skills conflict on handling the same scenario, this marks an underspecified boundary — a candidate location for a new mediating skill.

3.4 sim-conductor — Pre-Deployment Simulation (F4)

sim-conductor addresses F4 through structured simulation of non-owner harness adoption before cascade deployment.

Area A — External User Simulation: A cold-start external user persona is simulated — no author context, no shared vocabulary history, no session transcripts. The simulation attempts to trigger skills using natural vocabulary, reach expected outcomes via documented workflows, and identify vocabulary barriers for unfamiliar users.

Area B — Internal Consistency Simulation: Validates consistent harness operation across a multi-session arc, verifying that memory files, skill triggers, and agent dispatch rules remain self-consistent over time without human correction.

Cascade Gate: sim-conductor Area A must achieve $\geq 80\%$ skill reachability via natural vocabulary before cascade deployment is approved.

4. Evaluation

We applied all four methods to **forge-harness** [chrono-code, 2026], a meta-harness toolkit for Claude Code comprising 23 skills, 3 agents, and 2 plugin namespaces (fh-meta, fh-commons). Applying forge-harness methods to forge-harness itself constitutes a self-verification scenario (F1), addressed through external validation (Section 4.3).

4.1 steel-quench: Wave 1–3

The devil agent identified 10 findings:

Attack Angle	Grade	Finding
Documentation-code alignment	S	CI agent count used <code>find -type d</code> ; agents are <code>.md</code> files → CI always reported 0 agents
Documentation-code alignment	S	README referenced <code>~/PycharmProjects/</code> (internal path) in 8 locations
Self-referential structure	S	<code>plugin.json</code> version pinned to <code>0.0.0</code> with no CI enforcement
Self-referential structure	S	README self-marketing claims without external citation
Bus factor	A	All skills authored by single contributor; no cold-start validation
Self-referential structure	A	Author placeholder <code>your-name@example.com</code> in published config
Self-referential structure	A	No <code>engines.claudeCode</code> compatibility field
Existence justification	A	29 skill activations without explicit benefit-scaling justification
Platform obsolescence	B	No degradation path if Claude Code plugin API changes
Self-referential structure	B	Internal-only plugin referenced without external equivalent

All 4 S-grade blockers were immediately remediated. All 4 A-grade items resolved within the same session. Wave 3: zero new S-grade blockers. **Convergence achieved.**

4.2 source-grounding-audit Results

Applying source-grounding-audit to forge-harness SKILL.md content identified 3 phantom capability claims:

- **Capability phantom:** sim-conductor Area B documented as fully operational; cold-start execution confirmed the multi-session arc simulation had never been exercised end-to-end.
- **Measurement phantom:** README cited a token reduction figure without a measurement artifact. Claim removed; replaced with architectural description only.

- **Capability phantom:** verify-bidirectional described link-checking as automatic; inspection confirmed manual invocation is required per link pair.

Phantom rate before audit: 3/47 documented claims (6.4%). After remediation: 0/44 (0%). Three claims removed rather than fabricating evidence.

4.3 harvest-loop Results

Five external pull requests (#1–#5) were processed through harvest-loop. Synthesizer gate assignments:

PR	Pattern	Grade	Outcome
#1	CI agent counting fix + path neutralization	HIGH	Immediate integration
#2	arXiv study ingestion → frontier digest absorption	HIGH	README external validation block
#3	Internal placeholder removal	MED	Applied; no skill upgrade generated
#4	harness-evolver cross-audit → sister asset protocol	HIGH	SKILL.md positioning section added
#5	Regression guard + worktree isolation + numeric scoring	HIGH	Three SKILL.md upgrades (v1.2)

4.4 sim-conductor Results

sim-conductor Area A identified two vocabulary barriers before cascade deployment: (1) `/steel-quench` was not reachable by "adversarial review" or "devil's advocate check"; (2) `harvest-loop` was not triggered by "session cleanup" or "wrap up" phrasing. Both resolved by expanding trigger phrase lists. Post-fix: 26/26 skills (100%) reachable via natural vocabulary across 4 simulated external user profiles.

4.5 External Validation and Cascade Deployment

External validation was conducted through an independent GitHub account on a separate network with a fresh repository clone and no shared session context. This account submitted five PRs independently, identifying one additional defect (CI agent counting bug) that primary-environment validation had missed — confirming that external perspective surfaces blind spots not accessible from within the primary environment.

Beyond the evaluation environment, forge-harness was autonomously adopted by three additional practitioners. One practitioner applied steel-quench independently to a QA automation tool, identified two structural defects, and submitted corrections without author assistance. This non-owner autonomous operation constitutes observational validation that all four methods transfer without the originating author's presence.

4.6 Independent Architectural Convergence

Frontier search during the evaluation identified harness-evolver [Christi, 2026] — an independent Claude Code plugin implementing a 7-stage harness optimization loop with 6 agents. Cross-audit confirmed that harness-evolver and forge-harness independently converged on the same outer-loop architecture: field observation → adversarial critique → synthesis → integration → verification. The Stanford IRIS Lab Meta-Harness [arXiv:2603.28052] implements the same outer-loop principle at automated benchmark scale. Three independent implementations arriving at the same loop structure through different paths validates the architecture as a structural property of the harness engineering problem.

5. Discussion

5.1 Limitations

Single-project evaluation: All four methods are evaluated on forge-harness itself. Multi-project controlled evaluation is required to establish generalizability.

External validation independence: The "external" account used in Section 4.3 is operated by the same author from a different network context, not an independent practitioner. Truly independent external validation would require practitioners with no prior knowledge of forge-harness.

Phantom audit coverage: source-grounding-audit was applied to a subset of SKILL.md documentation; the 6.4% phantom rate is a partial-coverage estimate.

Cascade deployment: Section 4.5 reports observational adoption without controlled measurement of effect sizes or error rates relative to baselines.

Wave 4 coverage: Meta-aware adversary mode is documented in the framework but Wave 4 results are not reported in this evaluation.

Human-in-the-loop ceiling: forge-harness intentionally preserves human approval gates at each pipeline stage. This is a deliberate architectural choice — not a limitation — but practitioners requiring fully automated harness code search at benchmark scale should evaluate harness-evolver instead.

5.2 When to Apply Each Method

Situation	Recommended Method
Pre-deployment validation of a complex harness	steel-quench Wave 1–3
Harness content has grown through AI-generated documentation	source-grounding-audit
Working sessions generate learnings that are not persisted	harvest-loop
First non-owner adoption is imminent	sim-conductor Area A
Post-Wave-3 high-stakes release	steel-quench Wave 4

6. Conclusion

We presented forge-harness, a toolkit of four methods addressing the primary failure modes in AI collaboration harness engineering: steel-quench for adversarial structural validation, source-grounding-audit for phantom claim detection, harvest-loop for session-to-harness self-evolution, and sim-conductor for pre-deployment transfer validation. Applied to forge-harness itself, the methods resolved 10 structural defects, detected and removed 3 phantom claims, absorbed 5 external contributions as validated upgrades, and confirmed multi-user autonomous deployment — all within a single evaluation session. Independent architectural convergence across three implementations validates the outer-loop structure as a robust solution pattern.

The methods share a common design principle: each separates the roles of generator and evaluator that typically collapse in solo harness development. This separation is the structural mechanism by which the methods surface what informal self-evaluation cannot.

Artifact availability: <https://github.com/chrono-code/forge-harness>

References

- [arXiv:2604.14228] Chen, X. et al. "98.4% of Claude Code is Harness Infrastructure." VILA-Lab, 2026.
- [arXiv:2605.18747] [43 authors]. "Code as Agent Harness." 2026.
- [arXiv:2604.21003] Sylph.AI. "The Last Harness You'll Ever Build." 2026.
- [arXiv:2603.28052] Stanford IRIS Lab. "Meta-Harness: Automated Harness Optimization via Outer-Loop Execution." 2026.
- [Christi, 2026] Christi, R. harness-evolver. MIT License. <https://github.com/raphaelchristi/harness-evolver>, 2026.
- [Perez et al., 2022] Perez, E. et al. "Red Teaming Language Models with Language Models." arXiv:2202.03286.
- [Bai et al., 2022] Bai, Y. et al. "Constitutional AI: Harmlessness from AI Feedback." arXiv:2212.08073.
- [Maynez et al., 2020] Maynez, J. et al. "On Faithfulness and Factuality in Abstractive Summarization." ACL 2020.
- [Ji et al., 2023] Ji, Z. et al. "Survey of Hallucination in Natural Language Generation." ACM Computing Surveys, 2023.
- [Shinn et al., 2023] Shinn, N. et al. "Reflexion: Language Agents with Verbal Reinforcement Learning." NeurIPS 2023.
- [Madaan et al., 2023] Madaan, A. et al. "Self-Refine: Iterative Refinement with Self-Feedback." NeurIPS 2023.